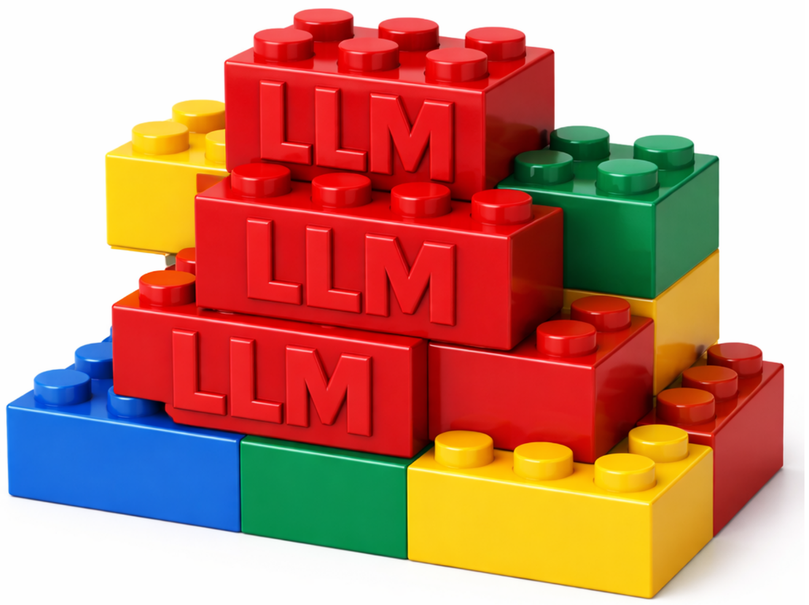
Panorama

Des usages concrets d’une LLM



Fabien Rohrer - Freelance - MSc in Computer Science, EPFL

Architectures d’orchestration

Chat

Typiquement chatGPT

L'appli construit à chaque tour un **prompt complet** composé de :

- **Preambule global (instructions)** : rôle, ton, règles, format de réponse attendu.
- **Historique de conversation** : une suite structurée de messages (User / Assistant).
- **Préambule de tour (avant chaque interaction)** : rappel de consignes, contraintes du moment (ex: "réponds en 5 puces", "sois concis", "langue: FR").
- Dernier message utilisateur.

L'appli impose la forme "chat" en alternant :

- un message `user` (question / consigne)
- puis un message `assistant` (réponse)

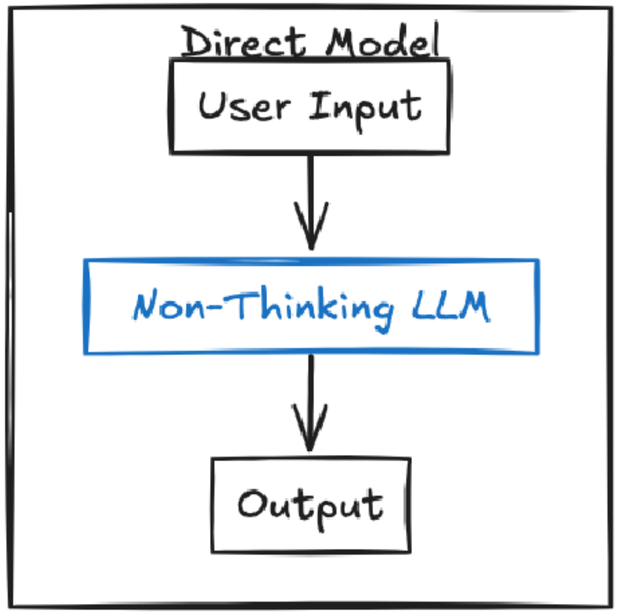
L'appli gère des aspects "produit" autour :

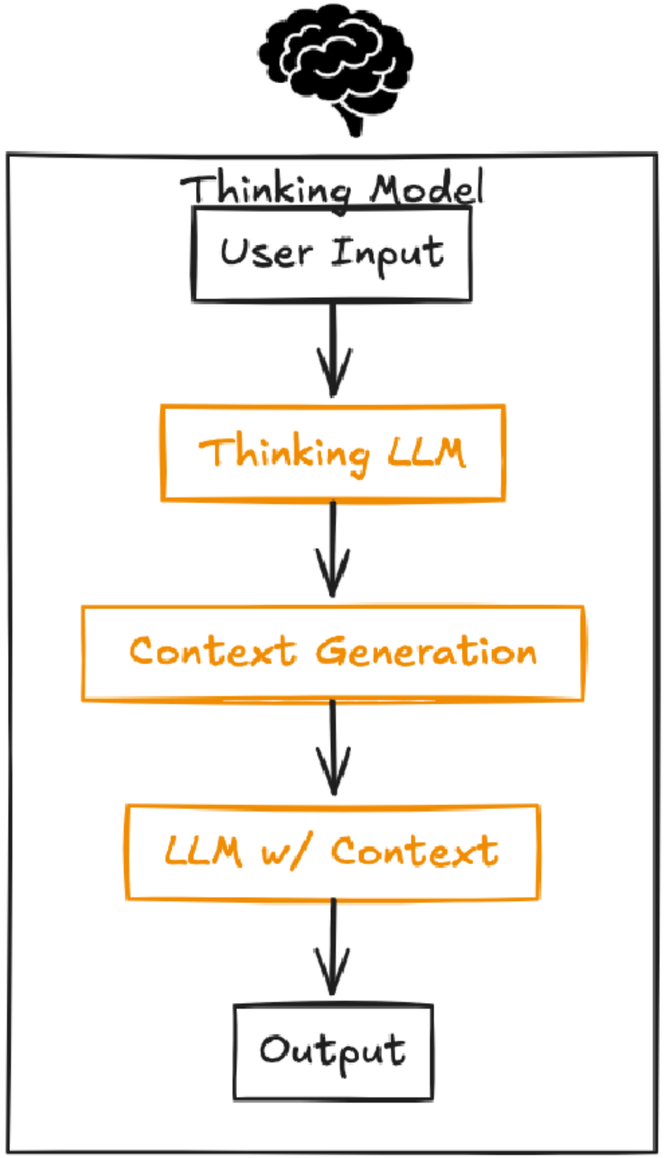
- **affichage** (streaming, markdown), **boutons** (copier), **reset** / nouveau chat
- **limites de contexte** (si trop long : tronquer ou résumer l'historique – optionnel mais fréquent)

Résultat : l'expérience "conversationnelle" vient surtout de **l'orchestration**, pas d'un modèle qui "discute tout seul".

Thinking

- Problèmes **multi-étapes** (planification, logique)
- **Analyse** et synthèse structurée
- **Vérification** / auto-contrôle (moins d'oublis)
- Augmente la probabilité d'une réponse **cohérente**





Coût

- Plus de tokens
- Plus lent

Outils / plugins

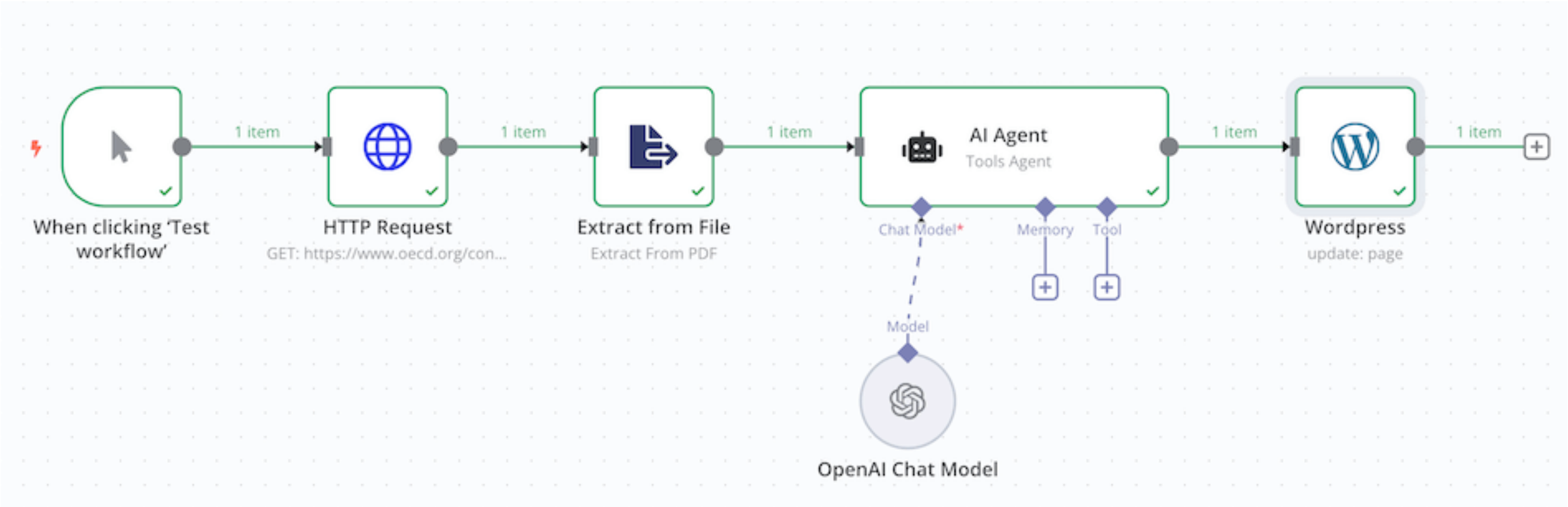
LLM + capacités externes

```
Utilisateur > Peux-tu chercher les tendances IA actuelles ?

IA > Je vais regarder ça pour toi. Un instant…

IA > {action: "web" ; parameter : "tendances IA"}

Outil > ${`action: web ; answer : "blah blah"`}

IA > Voilà, j'ai trouvé des articles sur les tendances IA de 2026. Voici quelques points
clés…

…
```

Multi modalités, Calculs, Actualités, Actions, etc.

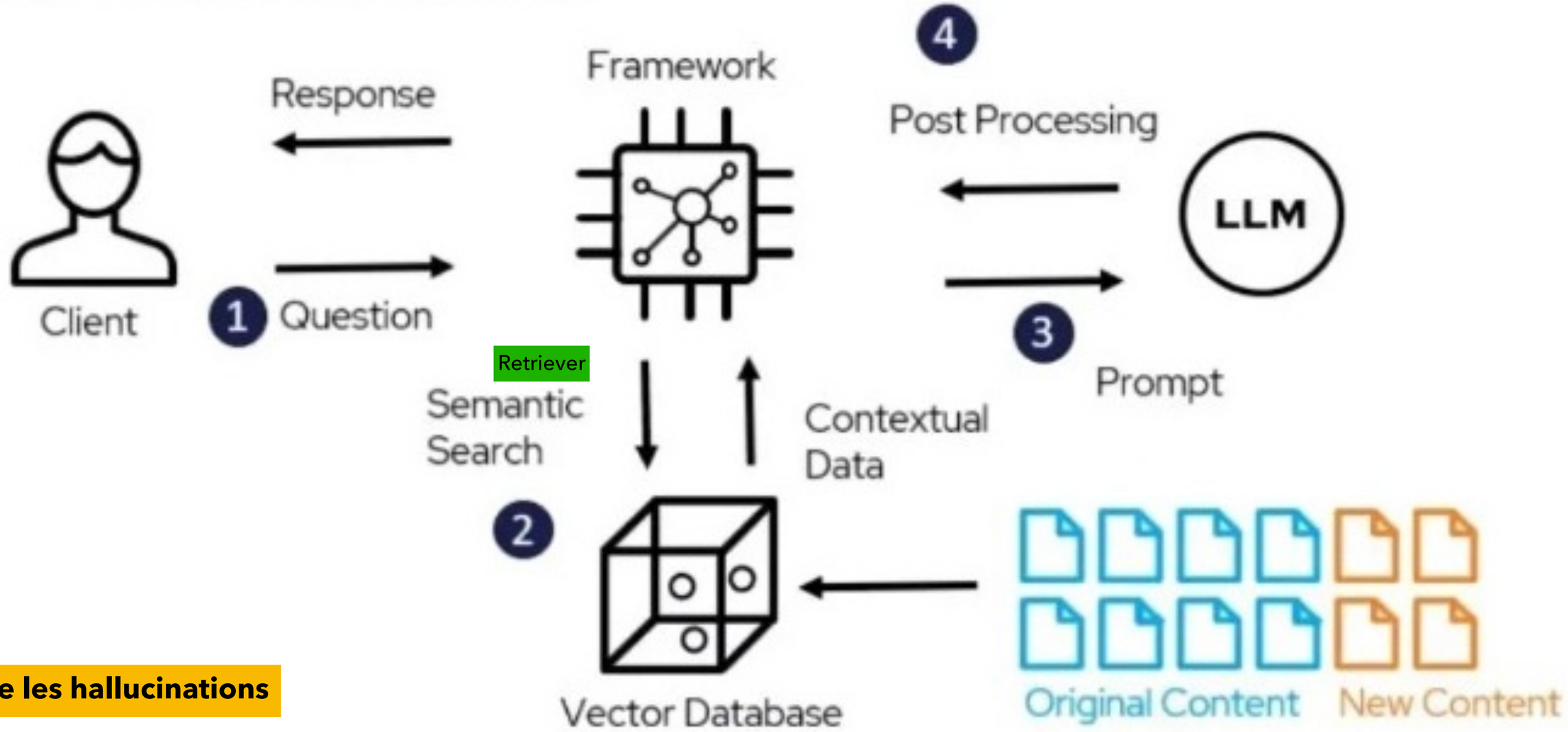
Automatisation

n8n, make



RAG

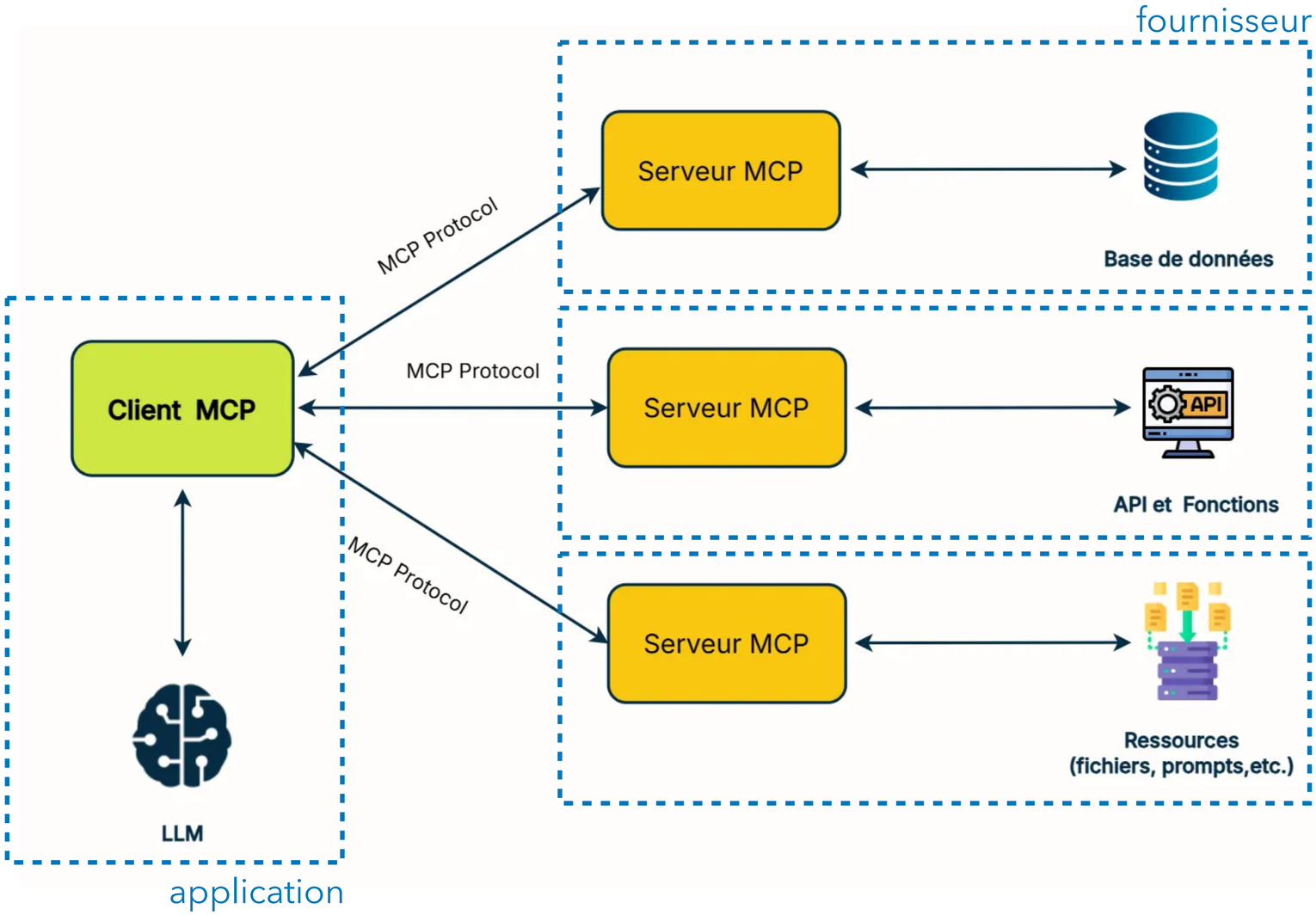
Retrieval Augmented Generation



Evite les hallucinations

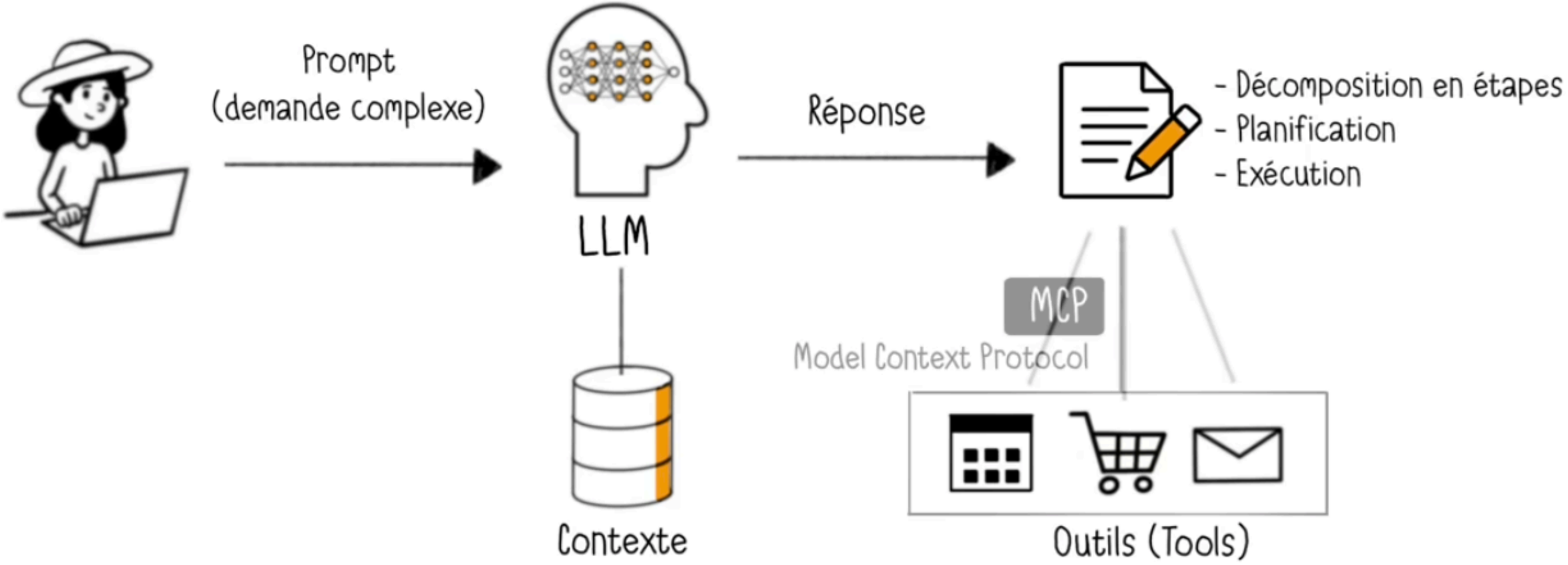
MCP

Model Context Protocol



Agent

Transforme une intension en une série d'action



IA Agentique

IA agentique

IA autonome, orientée action

Repose sur :

- Modèles Multimodaux pour la perception
- LLM pour le raisonnement
- Mémoire / RAG pour l'apprentissage continu
- Outils (Tools) pour l'exécution

Questions ?

Pour le code

Assistant du développeur

accélérer la génération de code

Générer : fonctions, squelettes, patterns “standards”

Refactor : simplifier, renommer, découper, clarifier

Tests : squelette JUnit/Jest, cas limites, data builders

Debug : expliquer une erreur, proposer hypothèses, pistes de logs

Doc : Javadoc/README, exemples d’usage, commentaires utiles

Compréhension: analyser, synthétiser, comprendre de vastes projets

Planification: organisation, découpage en tâches, etc.

- Intégration à l’IDE
- CLI

L'IA au service de l'équipe

standardiser, accélérer, fiabiliser

1) Qualité & standards (collectif)

- **Revue de code assistée** : détection d'odeurs, conventions, risques, sécurité basique
- **Alignement sur les patterns** : style guide, architecture, "do/don't" de l'équipe
- **Checklists de PR** : critères d'acceptation, points de vigilance, définition of done

2) Automatisation d'équipe (workflow)

- **Aide à la CI** : suggestions de tests manquants, scénarios de non-régression
- **Docs & changelog** : génération de notes de release, docs API, README à jour
- **Tri & pré-remplissage** : tickets, templates, description de bug, reproduction steps

3) Partage de connaissance & onboarding

- **Résumé de PR / modules** : "ce qui change" + impacts + zones touchées
- **Cartographie du code** : où modifier, dépendances, points d'entrée
- **Onboarding accéléré** : explications contextualisées + glossaire projet

Github

Questions ?